ЧАСТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА» (ЧПОУ «СКП»)

КУРСОВАЯ РАБОТА

Тема: «Проектирование и разработка модуля учета выдачи оборудования сотрудникам»

ПО ПМ.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

МДК.01.01 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил:
студент группы ИСиП-23/1
Пономарев Никита
Сергеевич

Руководитель:
Крючков Виталий
Владимирович

Сургут, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....6
1.1 Учёт выдачи оборудования сотрудникам.....6
1.2 Обзор аналогов.....7
1.3 Требования к модулю.....7
1.4 Выбор средств разработки.....9
Выводы по разделу 1.....9
2 ПРОЕКТИРОВАНИЕ МОДУЛЯ.....11
2.1 Модель данных.....11
2.2 Главное правило и определение держателя.....13
2.3 Архитектура модуля.....14
2.4 Алгоритм оформления выдачи.....15
Выводы по разделу 2.....16
3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МОДУЛЯ.....17
3.1 Справочники оборудования и сотрудников.....17
3.2 Выдача оборудования.....18
3.3 Возврат и состояние оборудования.....21
3.4 Оборудование у сотрудников и история операций.....22
3.5 Хранение данных и развёртывание.....24
3.6 Тестирование программного модуля.....25
Выводы по разделу 3.....29
ЗАКЛЮЧЕНИЕ.....31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....32
ПРИЛОЖЕНИЕ А. ЛИСТИНГ ТЕСТОВ ОПРЕДЕЛЕНИЯ ДЕРЖАТЕЛЯ ОБОРУДОВАНИЯ.....34
ПРИЛОЖЕНИЕ Б. ЛИСТИНГ ПРАВИЛ ПРЕДМЕТНОЙ ОБЛАСТИ.....36

ВВЕДЕНИЕ

В каждой организации сотрудникам выдаётся оборудование: ноутбуки, мониторы, телефоны, гарнитуры, измерительные приборы, инструмент. Оборудование стоит дорого, переходит от одного сотрудника к другому, ломается, ремонтируется и в конце концов списывается. Ответственный за оборудование должен в любой момент ответить на простые вопросы: у кого сейчас находится конкретный ноутбук, что числится за сотрудником, который увольняется, в каком состоянии единицу вернули после использования и когда её выдавали в последний раз.

Актуальность темы определяется тем, что учёт выдачи оборудования во многих небольших организациях и учебных заведениях ведётся в бумажном журнале или в электронной таблице. В таких записях легко «выдать» одну и ту же единицу двум сотрудникам, забыть отметить возврат или потерять сведения о неисправности. В результате при увольнении сотрудника оборудование не возвращается, неисправная техника снова уходит в работу, а история использования восстанавливается только по памяти. Требуется простой программный модуль, который хранит справочники, фиксирует каждую выдачу и каждый возврат с датой и состоянием и сам не допускает недопустимых операций.

Степень разработанности проблемы. Задачи учёта оборудования решаются как крупными учётными системами («1С:Бухгалтерия», «1С:Управление IT-отделом», системы класса ITSM), так и электронными таблицами. Первые требуют лицензий, настройки и обучения персонала и избыточны для подразделения с несколькими десятками единиц техники; вторые не защищают от ошибок ввода и не связывают записи между собой. Поэтому разработка небольшого специализированного модуля с жёсткими правилами проверки данных остаётся практически значимой задачей.

Объект исследования – процесс выдачи оборудования сотрудникам организации и его возврата.

Предмет исследования – программный модуль учёта выдачи оборудования сотрудникам, обеспечивающий регистрацию выдачи и возврата, учёт состояния оборудования и историю операций.

Цель работы – спроектировать и разработать на платформе .NET 10 модуль учёта выдачи оборудования сотрудникам, который ведёт справочники сотрудников и оборудования, регистрирует выдачу и возврат с датой операции и состоянием оборудования, показывает список оборудования, находящегося у сотрудников, и хранит историю операций.

Для достижения цели поставлены задачи:

– проанализировать предметную область и сформулировать требования к модулю;

– выбрать и обосновать средства разработки;

– спроектировать модель данных, архитектуру и алгоритм главной операции – выдачи;

– реализовать справочники, выдачу и возврат, учёт состояния и списание, список оборудования у сотрудников и историю операций;

– разработать автоматические тесты и выполнить ручные проверки всех обязательных функций;

– подготовить инструкцию запуска и инструкцию пользователя.

Методы исследования: анализ предметной области и аналогов, объектно-ориентированное проектирование, моделирование данных (ER-диаграмма), моделирование состояний объекта, модульное тестирование, ручное функциональное тестирование.

Теоретическая значимость работы состоит в формализации правил учёта выдачи оборудования: признак «у кого находится оборудование» определяется не отдельным полем, а вычисляется по истории операций, а условия допустимости выдачи и возврата записаны в виде проверяемых правил. Практическая значимость – модуль готов к использованию в небольшой организации или учебном заведении: он запускается одной командой, хранит данные в файле базы SQLite и не требует отдельного сервера базы данных.

Работа состоит из введения, трёх разделов, заключения, списка использованных источников и приложений. В первом разделе проанализирована предметная область и сформулированы требования, во втором описано проектирование модуля, в третьем – реализация, развёртывание и тестирование.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Учёт выдачи оборудования сотрудникам

Оборудование, которое организация передаёт сотрудникам для работы, относится к её имуществу. Федеральный закон «О бухгалтерском учёте» [1] требует, чтобы каждый факт хозяйственной жизни подтверждался первичным документом, а федеральный стандарт ФСБУ 6/2020 «Основные средства» [2] определяет порядок учёта основных средств. Трудовой кодекс Российской Федерации [3] устанавливает материальную ответственность работника за ущерб, причинённый имуществу работодателя. Поэтому для каждой единицы оборудования важно знать, кому и когда она была передана и в каком состоянии возвращена.

На практике процесс выглядит так. Ответственный за оборудование (системный администратор, заведующий хозяйством, офис-менеджер) ведёт перечень техники с инвентарными номерами. Когда сотруднику нужна единица оборудования, ответственный отмечает выдачу: что выдано, кому, когда. При возврате он осматривает оборудование, фиксирует его состояние и отмечает возврат. Неисправное оборудование отправляется в ремонт и не выдаётся до его окончания, а непригодное списывается. При увольнении сотрудника проверяется, что за ним не числится оборудование.

Без автоматизации в этом процессе возникают типичные проблемы:

– одна и та же единица может быть записана выданной двум сотрудникам, если ответственный забыл отметить возврат;

– состояние при возврате не фиксируется, и неисправная техника снова уходит в работу;

– чтобы узнать, что числится за сотрудником, нужно просматривать весь журнал;

– записи можно исправить задним числом, и история использования оборудования теряется;

– уволенному сотруднику по ошибке выдаётся оборудование, а за увольняющимся остаётся невозвращённое.

1.2 Обзор аналогов

Для сравнения рассмотрены три распространённых способа учёта: учётная система «1С», специализированные системы учёта IT-активов (класса ITSM) и электронная таблица. Сравнение по критериям, важным для задачи, приведено в таблице 1.

Таблица 1 – Сравнение способов учёта выдачи оборудования

Критерий	**«1С»**	**ITSM-система**	**Таблица Excel**	**Разрабатываемый модуль**
Кто сейчас пользуется оборудованием	да	да	вручную	да, вычисляется автоматически
Запрет повторной выдачи	настраивается	да	нет	да
Состояние при возврате	частично	да	вручную	да, обязательно
История операций	да	да	частично	да
Стоимость	платная лицензия	подписка	входит в офисный пакет	бесплатно
Сложность освоения	высокая	высокая	низкая	низкая
Работа без сервера	да (локальная установка)	нет	да	да

Из сравнения видно, что крупные системы полностью решают задачу, но избыточны и дороги для небольшого подразделения, а таблица не защищает от ошибок. Разрабатываемый модуль занимает промежуточное положение: он прост в освоении и развёртывании, но строго проверяет каждую операцию.

1.3 Требования к модулю

Требования сформулированы по заданию на курсовую работу: реализовать справочники сотрудников и оборудования, регистрацию выдачи и возврата, дату операции и состояние оборудования; предусмотреть список оборудования, находящегося у сотрудников, и историю операций. Функциональные требования приведены в таблице 2.

Таблица 2 – Функциональные требования к модулю

Код	Функция	Содержание
Ф1	Справочник оборудования	добавление, просмотр, изменение; поиск по наименованию, инвентарному и серийному номеру; фильтры по категории, состоянию, местонахождению
Ф2	Справочник сотрудников	добавление, просмотр, изменение, поиск; отметка увольнения вместо удаления
Ф3	Выдача	оборудование, сотрудник, дата операции, комментарий; запрет недопустимой выдачи
Ф4	Возврат	дата операции и состояние при возврате; сотрудник определяется автоматически
Ф5	Состояние оборудования	«Исправно», «Есть износ», «Требует ремонта»; изменение после ремонта; списание вместо удаления
Ф6	Оборудование у сотрудников	список выданного с датой выдачи и числом дней на руках; фильтры по сотруднику и подразделению
Ф7	История операций	все операции с фильтрами по оборудованию, сотруднику, виду операции и периоду; история в карточках

Ограничения предметной области (главные правила модуля) сведены в таблицу 3.

Таблица 3 – Ограничения предметной области

№	Ограничение	Пример неверного действия
1	Нельзя выдать оборудование, которое уже находится у сотрудника	выдать ноутбук ОВ-0001, числящийся за Кузнецовым А. И., другому сотруднику
2	Нельзя выдать оборудование, требующее ремонта, или списанное	выдать гарнитуру ОВ-0301 в состоянии «Требует ремонта»
3	Нельзя выдать оборудование уволенному сотруднику	выдать монитор Фёдоровой Е. П.
4	Нельзя вернуть оборудование, которое не выдано	оформить возврат монитора, лежащего на складе
5	Дата операции не может быть в будущем и раньше последней операции по оборудованию	возврат с датой раньше даты выдачи
6	При возврате в состоянии «Требует ремонта» неисправность описывается обязательно	возврат без комментария
7	Нельзя уволить сотрудника, у которого на руках оборудование, и списать выданное оборудование	отметить увольнение Кузнецова А. И. с двумя единицами на руках
8	Инвентарный и табельный номера уникальны, обязательные поля заполнены	повтор номера ОВ-0001, пустое наименование

Нефункциональные требования: интерфейс на русском языке; ошибки пользователя не приводят к аварийному завершению, а дают понятное сообщение; неизвестная запись даёт страницу «Запись не найдена» с кодом 404;

данные сохраняются между запусками; модуль запускается на компьютере с установленным .NET SDK без дополнительной настройки.

1.4 Выбор средств разработки

Модуль разработан как веб-приложение: им можно пользоваться из любого браузера в локальной сети без установки программы на каждое рабочее место. Выбранные средства и причины выбора приведены в таблице 4.

Таблица 4 – Средства разработки

Средство	Назначение	Причина выбора
C# и .NET 10	язык и платформа	строгая типизация, изучаются в курсе, актуальная версия с долгосрочной поддержкой
ASP.NET Core MVC [4]	веб-интерфейс и API	разделение на модели, представления и контроллеры; страницы и JSON-API в одном приложении
Entity Framework Core 10 [5, 6]	доступ к данным	работа с базой через классы C#, миграции для создания структуры базы
SQLite	хранение данных	база в одном файле, не требует сервера
xUnit [7]	автоматические тесты	стандартный инструмент тестирования для .NET
Bootstrap 5	оформление страниц	готовые стили форм и таблиц
Git, GitHub Actions [8]	хранение версий, сборка	история изменений по этапам, автоматическая сборка и тесты после каждой отправки

Платформа .NET 10 выбрана как актуальная версия на момент выполнения работы [9]. Шаблон ASP.NET Core MVC выбран потому, что модуль состоит из нескольких связанных разделов (справочники, операции, списки), а контроллеры позволяют держать обработку запросов отдельно от предметной логики.

Выводы по разделу 1

Проанализирован процесс выдачи и возврата оборудования и выявлены проблемы ручного учёта: повторная выдача, потеря сведений о состоянии, невозвращённое оборудование уволенных сотрудников. Сравнение аналогов показало потребность в простом модуле со строгими правилами. Сформулированы семь функциональных требований, восемь ограничений

предметной области и нефункциональные требования, выбраны средства разработки.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ

2.1 Модель данных

Модель данных состоит из трёх сущностей: оборудование (Equipment), сотрудник (Employee) и операция (EquipmentOperation). Оборудование и сотрудник – справочники, операция – журнал, в который добавляются записи о выдачах, возвратах, изменениях состояния и списаниях. Связи и поля показаны на рисунке 1 в соответствии с ER-диаграммой.

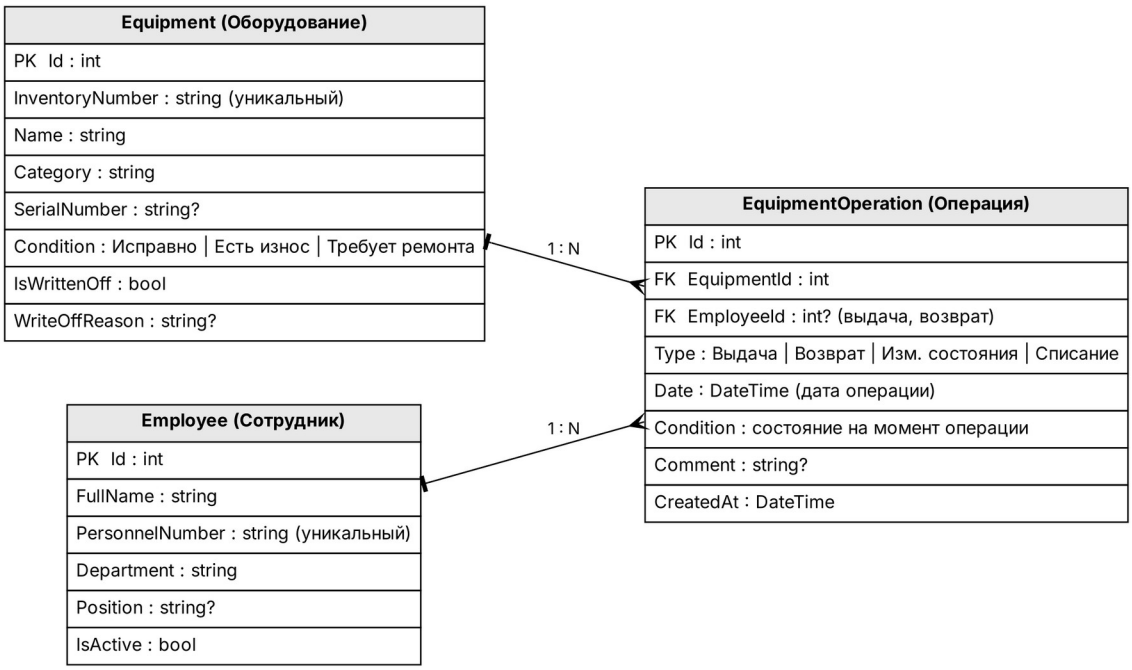


Рисунок 1 – ER-диаграмма модели данных

Описание сущностей приведено в таблице 5.

Таблица 5 – Сущности модели данных

Сущность	Поля	Ограничения
Equipment (оборудование)	Id, InventoryNumber, Name, Category, SerialNumber, Condition, IsWrittenOff, WriteOffReason	инвентарный номер обязателен и уникален; оборудование не удаляется, а списывается
Employee (сотрудник)	Id, FullName, PersonnelNumber, Department, Position, IsActive	ФИО, табельный номер и подразделение обязательны; табельный номер уникален; сотрудник не удаляется, а отмечается уволенным
EquipmentOperation (операция)	Id, EquipmentId, EmployeeId, Type, Date, Condition, Comment, CreatedAt	сотрудник обязателен для выдачи и возврата; запись не изменяется и не удаляется

Связь «оборудование – операции» имеет тип «один ко многим»: у единицы оборудования может быть сколько угодно операций, каждая операция относится ровно к одной единице. Связь «сотрудник – операции» также «один ко многим», но необязательная: у изменения состояния и списания сотрудника нет. Для обеих связей задано правило удаления Restrict – база не позволит удалить оборудование или сотрудника, у которых есть операции. Это защищает историю.

Сущности описаны классами C#. Класс оборудования приведён в листинге 1: атрибут [Display] задаёт русскую подпись поля на странице, [MaxLength] – длину столбца в базе.

Листинг 1 – Класс сущности «Оборудование» (Models/Equipment.cs, сокращённо)

```
public class Equipment
{
    public int Id { get; set; }

    [Display(Name = "Инвентарный номер")]
    [MaxLength(20)]
    public string InventoryNumber { get; set; } = string.Empty;

    [Display(Name = "Наименование")]
    [MaxLength(100)]
    public string Name { get; set; } = string.Empty;

    [Display(Name = "Состояние")]
    public EquipmentCondition Condition { get; set; } =
EquipmentCondition.Good;

    public bool IsWrittenOff { get; set; }          // списание вместо
удаления
    public string? WriteOffReason { get; set; }
    public List<EquipmentOperation> Operations { get; set; } = new();   //
связь 1 — N
}
```

Вид операции хранится перечислением OperationType (Issue – выдача, Return – возврат, ConditionChange – изменение состояния, WriteOff – списание), состояние – перечислением EquipmentCondition (Good – «Исправно», Worn – «Есть износ», NeedsRepair – «Требует ремонта»). В операции записывается состояние оборудования на момент операции, поэтому по истории видно, в каком виде единица была выдана и возвращена.

В модели нет поля «у кого сейчас оборудование». Это принципиальное проектное решение, которое описано в следующем подразделе.

2.2 Главное правило и определение держателя

Главное правило модуля: оборудование нельзя выдать, если оно уже находится у сотрудника, требует ремонта или списано, а также нельзя выдать его уволенному сотруднику. Чтобы проверить первое условие, нужно знать текущего держателя единицы. Если хранить его в отдельном поле, это поле может разойтись с журналом операций (например, при ошибке в коде или ручном исправлении базы). Поэтому держатель не хранится, а вычисляется по истории: операции выдачи и возврата данной единицы просматриваются в порядке даты (при одинаковой дате – в порядке создания), и держатель определяется по формуле (1):

$$H = E(\text{последней выдачи}),\ \text{если после неё нет возврата; иначе } H = \varnothing \qquad (1)$$

где H – текущий держатель оборудования ($\varnothing$ – оборудование на складе);

E(последней выдачи) – сотрудник, указанный в последней операции выдачи.

Выдача единицы оборудования сотруднику разрешается при одновременном выполнении условий (2):

$$H = \varnothing \ \wedge\ \neg\text{Списано} \ \wedge\ \text{Состояние} \neq \text{«Требует ремонта»} \ \wedge\ \text{Сотрудник работает} \qquad (2)$$

Дата d новой операции проверяется по условию (3):

$$d_{посл} \le d \le d_{сег} \qquad (3)$$

где $d_{посл}$ – дата последней операции по этой единице оборудования;

$d_{сег}$ – сегодняшняя дата.

Из условия (3) следует, что возврат не может быть оформлен раньше выдачи, а операции по одной единице выстраиваются во времени без противоречий. Граничный случай – операция в тот же день, что и предыдущая, – допустим: оборудование можно вернуть утром и в тот же день выдать другому сотруднику.

Возможные состояния единицы оборудования и переходы между ними показаны на рисунке 2. Каждая стрелка соответствует одной операции модуля; переходы, которых нет на схеме (например, выдача из состояния «Требует ремонта»), модуль отклоняет.

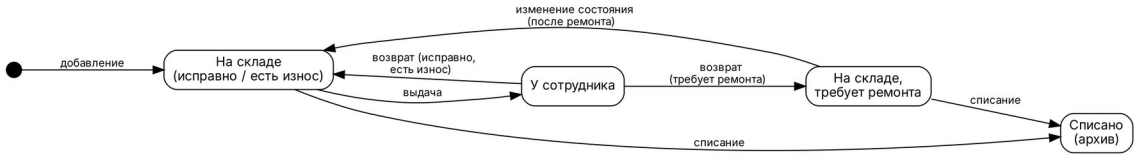


Рисунок 2 – Диаграмма состояний единицы оборудования

2.3 Архитектура модуля

Модуль построен по многослойной архитектуре (рисунок 3). Каждый слой выполняет свою задачу и обращается только к нижележащему.

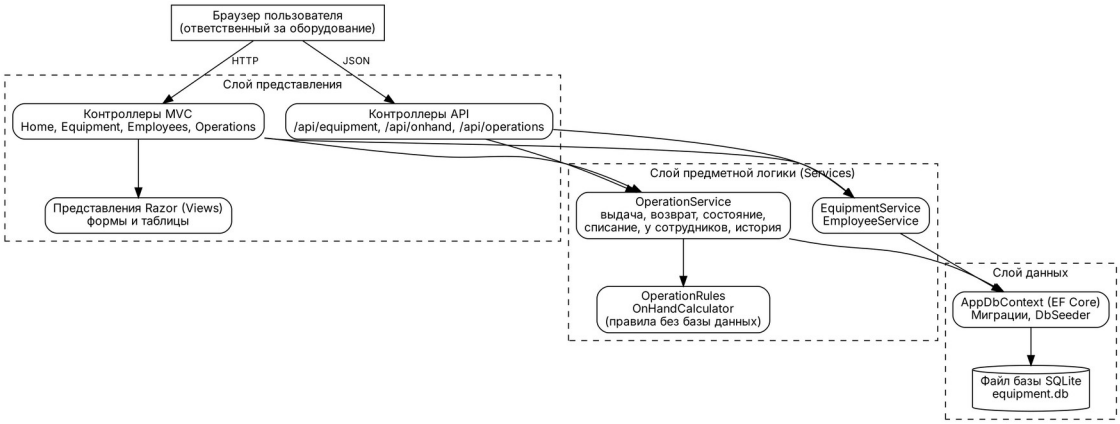


Рисунок 3 – Архитектура модуля

Слой представления – контроллеры и представления Razor. Контроллер принимает HTTP-запрос, вызывает метод сервиса и выбирает страницу для ответа; в нём нет ни правил предметной области, ни обращений к базе. Слой предметной логики – классы папки Services: OperationService (выдача, возврат, изменение состояния, списание, список у сотрудников, история), EquipmentService и EmployeeService (справочники), статические классы OperationRules (правила) и OnHandCalculator (определение держателя по формуле (1)). Слой данных – контекст AppDbContext, миграции и начальные учебные данные (DbSeeder).

Сервисы возвращают результат в виде объекта OperationResult с видом ошибки (неверные данные, запись не найдена, конфликт с правилами) и текстом сообщения. Ошибка пользователя – это не исключение, а обычный результат с понятным текстом на русском языке, который контроллер показывает на странице, а API возвращает с кодом 400, 404 или 409.

Варианты использования модуля показаны на рисунке 4. Пользователь модуля – ответственный за оборудование; сотрудники сами в модуль не входят, они получают и возвращают оборудование через ответственного.



Рисунок 4 – Диаграмма вариантов использования

2.4 Алгоритм оформления выдачи

Выдача – главная операция модуля. Порядок проверок показан на блок-схеме в соответствии с рисунком 5 (оформлена по ГОСТ 19.701-90 [10]). Сначала проверяются данные, не требующие обращения к базе, затем существование оборудования и сотрудника, дата и только после этого – держатель и состояние. Такой порядок позволяет как можно раньше отклонить неверный запрос.

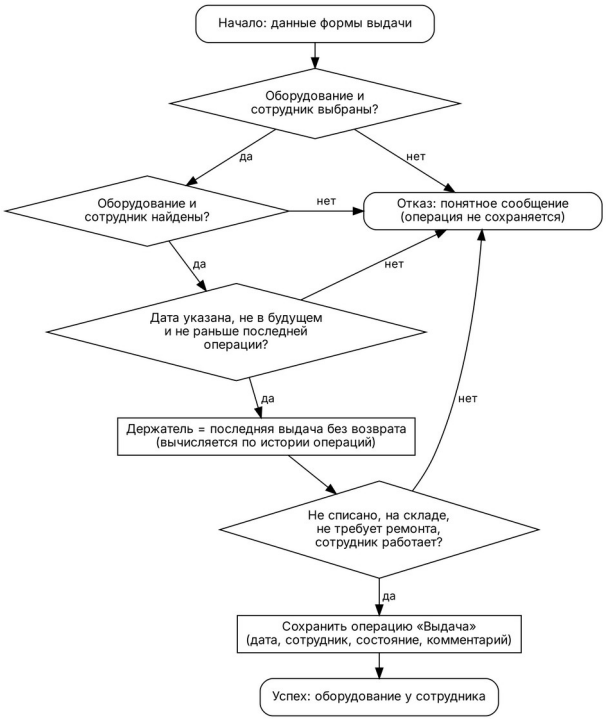


Рисунок 5 – Блок-схема алгоритма оформления выдачи

Проверка держателя и сохранение выдачи выполняются внутри критической секции (оператор lock). Если два пользователя одновременно оформят выдачу одной единицы, вторая операция начнёт проверку только после сохранения первой, увидит, что оборудование уже выдано, и будет отклонена.

Выводы по разделу 2

Спроектирована модель данных из трёх сущностей со связями «один ко многим». Текущий держатель оборудования не хранится, а вычисляется по истории операций по формуле (1), что исключает расхождение с журналом. Условия выдачи и проверки даты записаны в виде формул (2) и (3), построена диаграмма состояний единицы оборудования. Выбрана трёхслойная архитектура с отделением предметной логики от контроллеров, построены диаграмма вариантов использования и блок-схема главной операции с защитой от одновременных выдач.

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 Справочники оборудования и сотрудников

Справочник оборудования (рисунок 6) показывает все единицы с инвентарным номером, категорией, состоянием и местонахождением: «на складе» или «у» конкретного сотрудника с датой выдачи. Поиск выполняется по наименованию, инвентарному и серийному номеру без учёта регистра, фильтры – по категории, состоянию и местонахождению; списанное оборудование показывается по отдельному флажку.

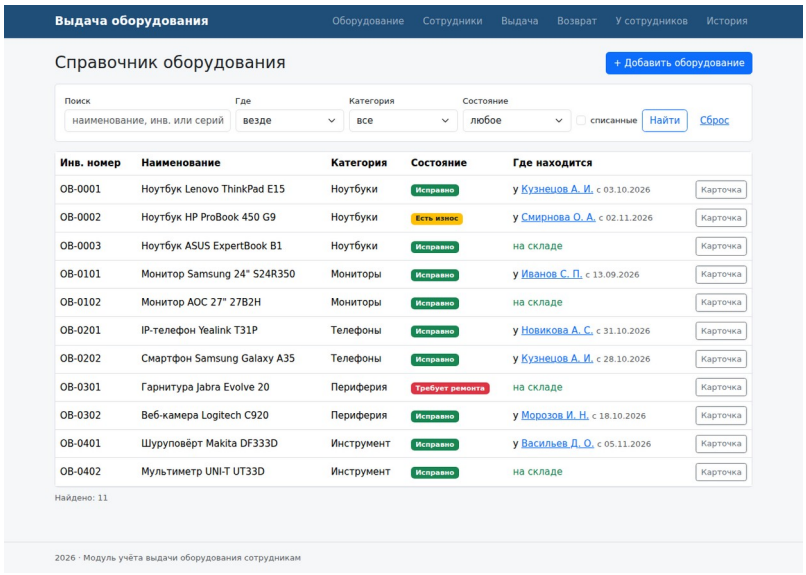


Рисунок 6 – Справочник оборудования

Результаты проверок справочника: Строк в справочнике: 11; поиск «ноутбук» – Найдено: Ноутбук Lenovo ThinkPad E15; Ноутбук HP ProBook 450 G9; Ноутбук ASUS ExpertBook B1; фильтр «у сотрудников» – У сотрудников: 7 ед.; фильтр по состоянию – Требует ремонта: Гарнитура Jabra Evolve 20.

При добавлении и изменении сервис EquipmentService приводит данные к единому виду (инвентарный номер – в верхний регистр, лишние пробелы удаляются) и проверяет их. Повтор номера отклоняется без учёта регистра: при вводе «ob-0001» получено сообщение «Оборудование с инвентарным номером OB-0001 уже есть в справочнике», при пустом наименовании – «Укажите наименование».

Справочник сотрудников хранит ФИО, табельный номер, подразделение и должность; в списке видно, сколько единиц оборудования у каждого на руках. Сотрудник не удаляется, а отмечается уволенным. Уволить сотрудника, за которым числится оборудование, нельзя (рисунок 7): получено сообщение «У сотрудника Кузнецов А. И. на руках оборудование (2 ед.). Сначала оформите возврат». Карточка сотрудника показывает оборудование на руках и историю его операций.

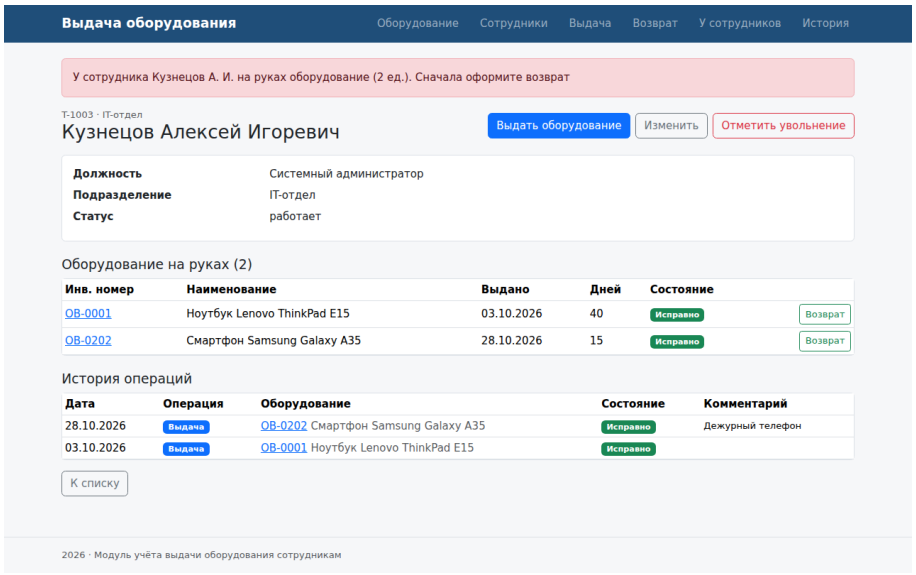


Рисунок 7 – Карточка сотрудника и отказ в увольнении

3.2 Выдача оборудования

Форма выдачи предлагает только то оборудование, которое можно выдать (на складе, не требует ремонта, не списано), и только работающих сотрудников; дата по умолчанию – сегодняшняя. Проверка в списке: ОВ-0001 в списке выдачи: нет; ОВ-0301 (требует ремонта): нет. Ограничение списка – удобство, а не защита: все правила повторно проверяются на сервере, поэтому подмена значения в форме или запрос через API не позволяют их обойти.

Ключевая часть метода выдачи приведена в листинге 2. Определение держателя и проверка правила выполняются под блокировкой, операция сохраняется только при отсутствии ошибок.

Листинг 2 – Выдача оборудования (Services/OperationService.cs, фрагмент)

```
lock (OperationLock)
{
    var equipment = _db.Equipment.FirstOrDefault(e => e.Id ==
form.EquipmentId);
    if (equipment == null) return OperationResult.NotFound("Оборудование не
найдено");
    var employee = _db.Employees.FirstOrDefault(e => e.Id ==
form.EmployeeId);
    if (employee == null) return OperationResult.NotFound("Сотрудник не
найден");

    var dateError = OperationRules.ValidateDate(form.Date, Today,
LastOperationDate(equipment.Id));
    if (dateError != null) return OperationResult.Invalid(dateError);

    var ruleError = OperationRules.CheckIssue(equipment, employee,
CurrentIssue(equipment.Id));
    if (ruleError != null) return OperationResult.Conflict(ruleError);

    var op = new EquipmentOperation { EquipmentId = equipment.Id, EmployeeId
= employee.Id,
        Type = OperationType.Issue, Date = form.Date!.Value.Date, Condition =
equipment.Condition,
        Comment = Clean(form.Comment), CreatedAt =
_clock.GetLocalNow().DateTime };
    _db.Operations.Add(op);
    _db.SaveChanges();
    return OperationResult.Ok(op.Id, $"Оборудование «{equipment.Name}» ...
выдано сотруднику ...");
}
```

Правило выдачи (условие (2)) записано в методе OperationRules.CheckIssue (листинг 3). Метод не обращается к базе, поэтому проверяется автоматическими тестами отдельно от интерфейса.

Листинг 3 – Правило выдачи (Services/OperationRules.cs)

```
public static string? CheckIssue(Equipment equipment, Employee employee,
EquipmentOperation? currentIssue)
{
    if (equipment.IsWrittenOff)
        return $"Оборудование «{equipment.Name}» списано — выдача
невозможна";
    if (currentIssue != null)
    {
        var holder = currentIssue.Employee != null ?
NameFormat.Short(currentIssue.Employee.FullName) : "другого сотрудника";
        return $"Оборудование «{equipment.Name}» уже выдано: находится у
{holder} с {currentIssue.Date:dd.MM.yyyy}. Сначала оформите возврат";
    }
    if (equipment.Condition == EquipmentCondition.NeedsRepair)
        return $"Оборудование «{equipment.Name}» требует ремонта — выдача
запрещена до ремонта";
    if (!employee.IsActive)
        return $"Сотрудник {NameFormat.Short(employee.FullName)} уволен —
выдача невозможна";
    return null;
}
```

Пример успешной выдачи: ноутбук ОВ-0003 выдан методисту Поповой М. В. (рисунок 8). Получено сообщение и новое местонахождение: «Оборудование «Ноутбук ASUS ExpertBook B1» (ОВ-0003) выдано сотруднику Попова М. В.; где находится: у Попова Мария Викторовна выдано 12.11.2026, на руках 0 дн.».

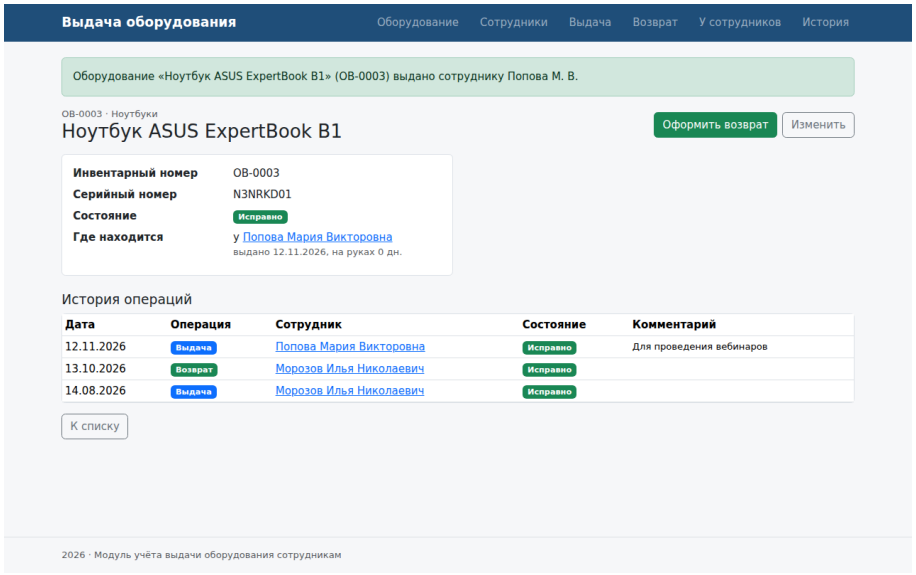


Рисунок 8 – Карточка оборудования после успешной выдачи

Пример отклонения по главному правилу: в форму подставлен ноутбук ОВ-0001, который уже числится за Кузнецовым А. И. (рисунок 9). Сервер отклонил операцию: «Оборудование «Ноутбук Lenovo ThinkPad E15» уже выдано: находится у Кузнецов А. И. с 03.10.2026. Сначала оформите возврат». Через API получены отказы: выдача оборудования, требующего ремонта, – «HTTP 409: Оборудование «Гарнитура Jabra Evolve 20» требует ремонта — выдача запрещена до ремонта»; уволенному сотруднику – «HTTP 409: Сотрудник Фёдорова Е. П. уволен — выдача невозможна»; списанного – «HTTP 409: Оборудование «Проектор Epson EB-E01» списано — выдача невозможна». Выдача без сотрудника даёт сообщение «Выберите сотрудника», с датой завтрашнего дня – «Дата операции не может быть в будущем».

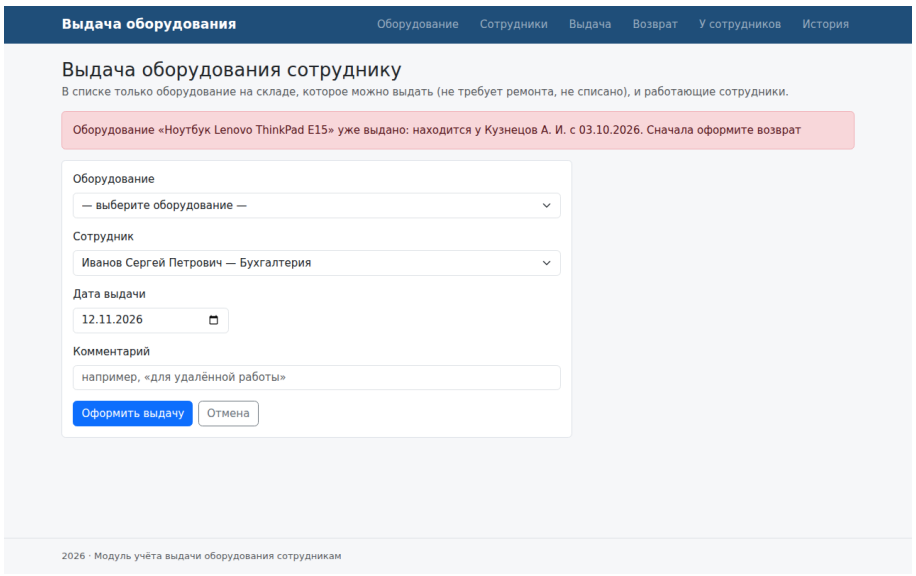


Рисунок 9 – Отклонение выдачи уже выданного оборудования

3.3 Возврат и состояние оборудования

При возврате пользователь выбирает выданное оборудование (в списке указано, у кого оно и с какой даты), дату и состояние. Сотрудник не выбирается – он подставляется автоматически по последней выдаче, поэтому вернуть оборудование «не от того» сотрудника невозможно. Состояние единицы в справочнике обновляется по факту возврата.

Если оборудование возвращается в состоянии «Требует ремонта», комментарий с описанием неисправности обязателен (рисунок 10): без него получено сообщение «Опишите неисправность в комментарии: оборудование возвращается в состоянии «Требует ремонта»». С комментарием возврат выполнен: «Оборудование «Ноутбук HP ProBook 450 G9» возвращено от Смирнова О. А., состояние: Требует ремонта. Выдача заблокирована до ремонта; где находится: на складе». После такого возврата выдача блокируется до ремонта: «HTTP 409: Оборудование «Ноутбук HP ProBook 450 G9» требует ремонта — выдача запрещена до ремонта».

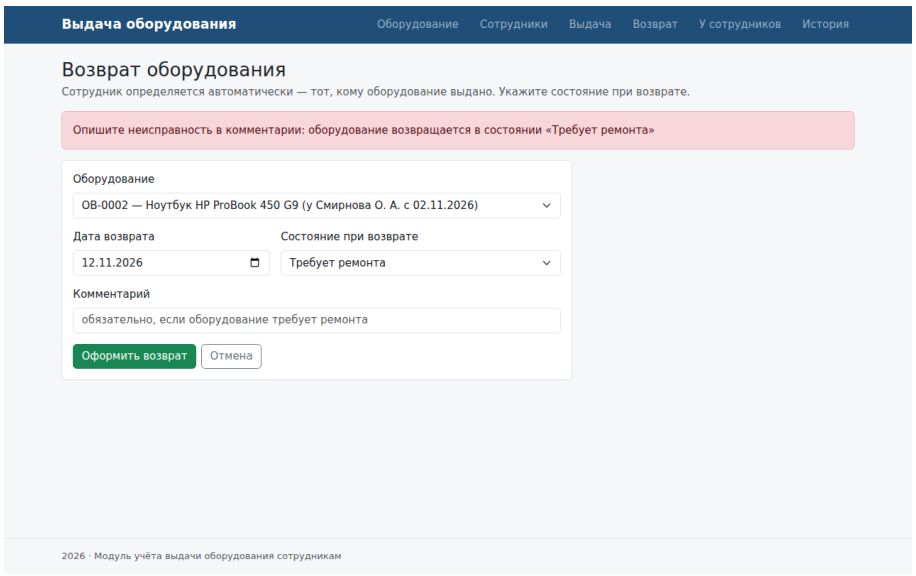


Рисунок 10 – Отклонение возврата без описания неисправности

Возврат оборудования, которое находится на складе, отклоняется: «HTTP 409: Оборудование «Монитор AOC 27" 27B2H» не числится выданным — возвращать нечего». Возврат с датой раньше выдачи нарушает условие (3): «HTTP 400: Дата операции не может быть раньше последней операции по этому оборудованию (05.11.2026)».

После ремонта ответственный меняет состояние в карточке оборудования с обязательным указанием причины; изменение записывается в историю отдельной операцией. Проверка: «Состояние изменено: Исправно», после чего выдача снова разрешена: «HTTP 201: Оборудование «Ноутбук HP ProBook 450 G9» (OB-0002) выдано сотруднику Морозов И. Н.».

Списание выполняется вместо удаления: запись остаётся в справочнике с причиной списания, а все её операции – в истории. Списать можно только оборудование на складе (Форма списания у выданного: нет); проверка списания со склада: «Оборудование «Мультиметр UNI-T UT33D» списано; где находится: Списано Сломан переключатель режимов».

3.4 Оборудование у сотрудников и история операций

Список оборудования, находящегося у сотрудников (рисунок 11), строится методом OnHandCalculator.CurrentIssues по формуле (1): для каждой выданной единицы показываются сотрудник, подразделение, дата выдачи и число дней на

руках. Список фильтруется по сотруднику, подразделению и оборудованию; из каждой строки можно сразу оформить возврат. Проверки: всего Строк: 8; фильтр по подразделению – IT-отдел: OB-0001; OB-0202; OB-0002; OB-0302.

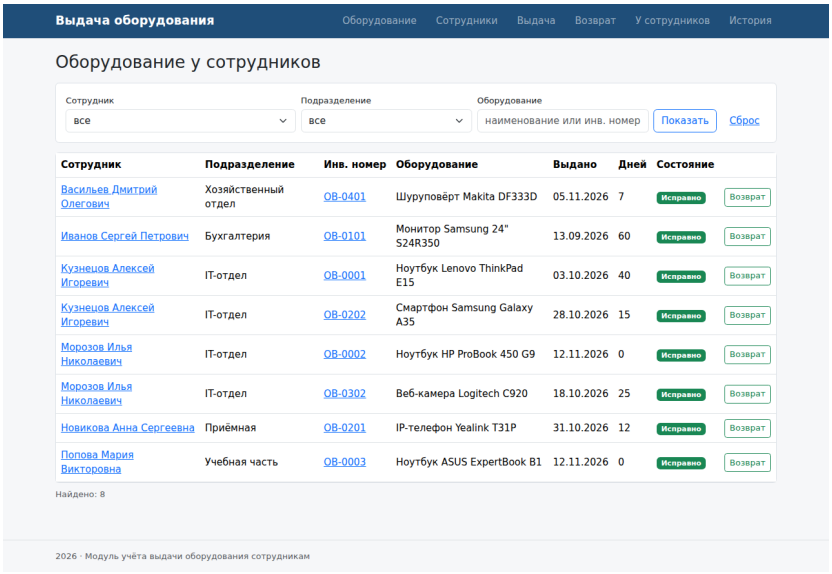


Рисунок 11 – Оборудование у сотрудников

Вычисление держателя приведено в листинге 4. Операции перебираются по дате и номеру: выдача записывает единицу в словарь текущих выдач, возврат удаляет её оттуда.

Листинг 4 – Определение текущих выдач (Services/OnHandCalculator.cs)

```
public static Dictionary<int, EquipmentOperation>
CurrentIssues(IEnumerable<EquipmentOperation> operations)
{
    var result = new Dictionary<int, EquipmentOperation>();
    foreach (var op in operations.OrderBy(o => o.Date).ThenBy(o => o.Id))
    {
        if (op.Type == OperationType.Issue)
            result[op.EquipmentId] = op;
        else if (op.Type == OperationType.Return)
            result.Remove(op.EquipmentId);
    }
    return result;
}
```

История операций (рисунок 12) показывает все операции, новые сверху, с фильтрами по оборудованию, сотруднику, виду операции и периоду; та же таблица выводится в карточках оборудования и сотрудника. Проверки: по ноутбуку OB-0002 – Найдено операций: 6; только возвраты – Найдено операций: 6; за последние 14 дней – Найдено операций: 9; при начале периода позже окончания – «Начало периода не может быть позже окончания».

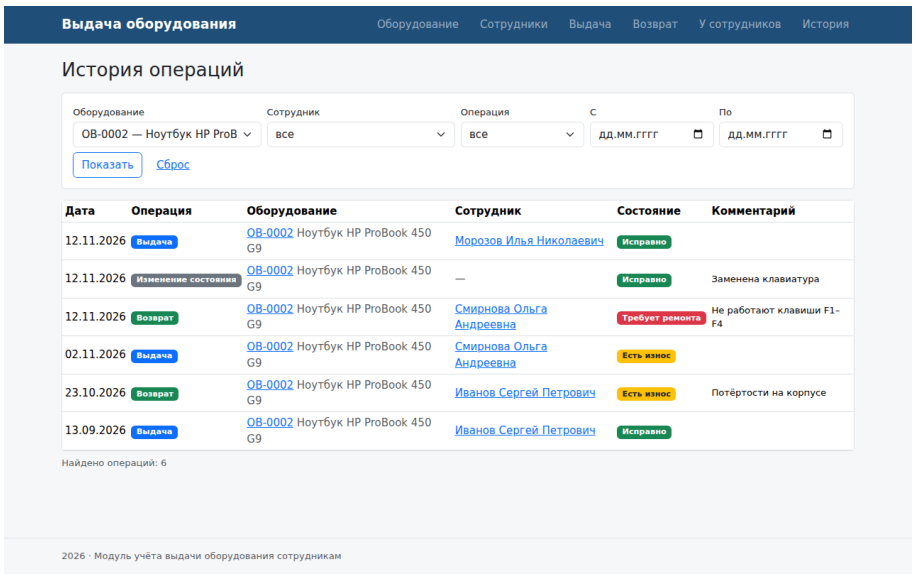


Рисунок 12 – История операций по единице оборудования

Главная страница показывает сводку: В учёте: 11, на складе: 4, выдано: 7, требует ремонта: 1.

3.5 Хранение данных и развёртывание

База SQLite создаётся миграцией InitialCreate. При запуске приложение вызывает метод Database.Migrate(), который создаёт файл equipment.db и таблицы, если их ещё нет, а затем DbSeeder добавляет учебные данные: 12 единиц оборудования, 8 сотрудников и 18 операций. Даты учебных операций отсчитываются от дня первого запуска, поэтому история и число дней на руках выглядят правдоподобно в любой день. Если оборудование в базе уже есть, учебные данные повторно не добавляются, и изменения пользователя сохраняются между запусками. Чтобы вернуть учебную базу в исходное состояние, достаточно остановить приложение и удалить файл equipment.db.

Сохранение данных проверено перезапуском приложения (рисунок 13): OB-0601 найден: да (Планшет Huawei MatePad 11); OB-0003 после перезапуска: у Попова Мария Викторовна выдано 12.11.2026, на руках 0 дн..

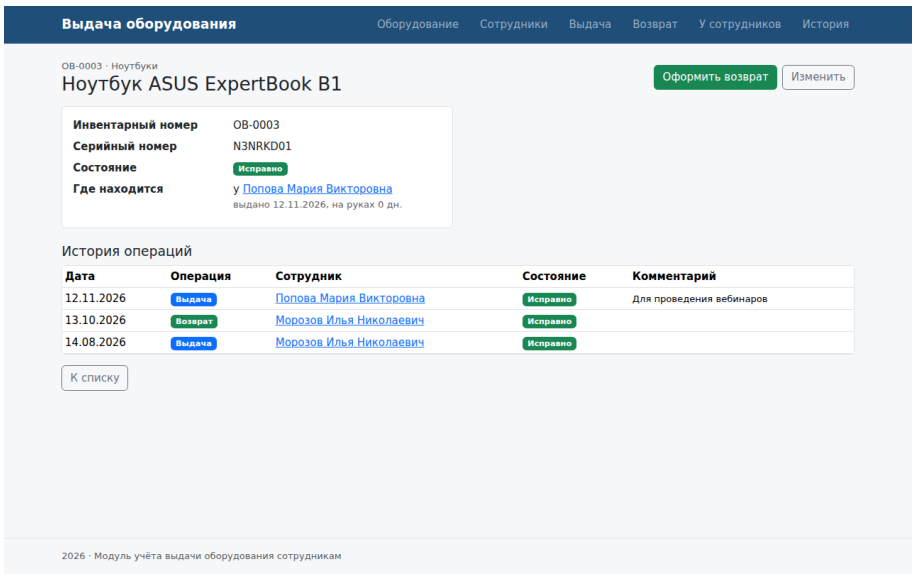


Рисунок 13 – Данные после перезапуска приложения

Для развёртывания нужен только .NET SDK 10: пакеты NuGet скачиваются автоматически при первой сборке. Порядок запуска описан в файле README.md проекта и в приложении «Инструкция запуска». Исходный код хранится в репозитории Git; каждый этап работы зафиксирован отдельными коммитами. После каждой отправки изменений на GitHub автоматически выполняются сборка, все тесты и обход основных страниц (GitHub Actions).

3.6 Тестирование программного модуля

Подход к тестированию. Автоматическими тестами проверяется предметная логика, которую можно проверить отдельно от интерфейса: определение держателя, правила выдачи и возврата, проверка даты, списание и изменение состояния, справочники, список у сотрудников, история, сводка и сохранение данных. Тесты вызывают методы сервисов напрямую и используют базу SQLite в оперативной памяти: для каждого теста создаётся новая пустая база. Сегодняшняя дата задаётся «остановленными часами» (класс TimeProvider), поэтому проверки «дата в будущем» и «дней на руках» не зависят от дня запуска тестов.

Каждый тест построен по схеме «подготовка – действие – проверка». В листинге 5 приведён тест главного правила: повторная выдача уже выданного оборудования отклоняется, операция не сохраняется, держатель не меняется.

Листинг 5 – Тест главного правила (EquipmentIssue.Tests/IssueTests.cs)

```
[Fact]
public void Выдача_УжеВыдано_Отказ409ИОперацияНеСохранена()
{
    using var env = new TestEnv();
    var eq = env.AddEquipment();
    var first = env.AddEmployee();
    var second = env.AddEmployee("Смирнова Ольга Андреевна", "Т-1002");
    env.Issue(eq.Id, first.Id, daysAgo: 3);

    var result = env.Issue(eq.Id, second.Id);

    Assert.Equal(ErrorKind.Conflict, result.Kind);
    Assert.Contains("уже выдано: находится у Иванов С. П. с 09.11.2026",
result.Error);
    Assert.Equal(1, env.OperationCount());
    Assert.Equal(first.Id, env.Ops(s => s.CurrentIssue(eq.Id))!.EmployeeId);
}
```

Тестовый проект EquipmentIssue.Tests содержит 64 тестовых метода; с учётом параметризованных тестов (Theory) выполняется 69 проверок. Состав тестов приведён в таблице 6.

Таблица 6 – Состав автоматических тестов

Тестовый класс	Функции	Проверок	Что проверяется	Результат
OnHandCalculatorTests	Ф6, Ф7	12	держатель без операций, после выдачи, после возврата, повторная выдача, порядок по дате, возврат и выдача в один день, текущие выдачи нескольких единиц, дни на руках	пройдены
OperationRulesTests	Ф3, Ф4, Ф5	20	дата: не указана, сегодня, завтра, раньше последней операции, в день последней; выдача: исправное, с износом, уже выдано, требует ремонта, списано, уволенному; возврат; длина комментария; краткое ФИО	пройдены
IssueTests	Ф3	12	успешная выдача; главное правило; ремонт, списание, уволенный; без оборудования и сотрудника; неизвестный Id; дата в будущем и раньше возврата; выдача в день возврата	пройдены
ReturnTests	Ф4, Ф5	6	успешный возврат и обновление состояния; возврат не выданного; «требует ремонта» без комментария и с ним; без состояния; дата раньше выдачи	пройдены
WriteOffAndConditionTests	Ф5	7	списание со склада и запрет выдачи; списание выданного; короткая причина; повторное списание; состояние после ремонта; у выданного; без комментария	пройдены
CatalogAndListsTests	Ф1, Ф2, Ф6, Ф7	12	повтор номера без учёта регистра; пустое наименование; поиск по-русски; местонахождение; увольнение с оборудованием и после возврата; повтор табельного номера; список у сотрудников; история; сводка; сохранение; учебные данные	пройдены

Тесты запущены командой dotnet test на сервере GitHub Actions (Ubuntu, .NET 10). Результат: всего 69 проверок, пройдено 69, не пройдено 0 (рисунок 14). Тесты определения держателя приведены в приложении А.

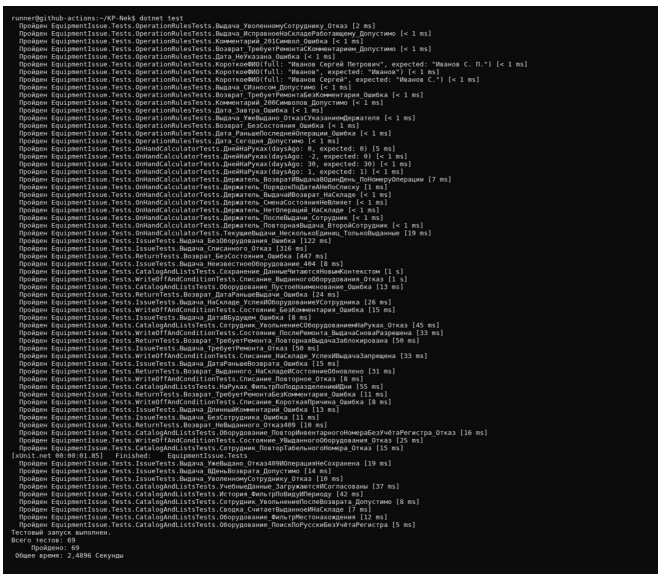


Рисунок 14 – Результаты выполнения автоматических тестов

Ручные проверки. Сценарии, связанные с интерфейсом, API и перезапуском приложения, проверены в браузере. Проверки выполнялись автоматизированным браузером (Google Chrome под управлением Playwright) по сценарию из файла tools/screenshots.py: сценарий выполнял действия пользователя, читал со страницы фактически появившееся сообщение и сохранял снимок экрана. Фактические результаты – это тексты, прочитанные со страниц и из ответов API, а не ожидаемые значения; они приведены в подразделах 3.1–3.5 и в отдельном документе «Таблица проверок». Ручные проверки API выполняются также по файлу EquipmentIssue.http.

Развёртывание из архива проверено так: архив проекта, созданный командой git archive, распакован в новую папку, после чего выполнены шаги инструкции запуска – сборка, автоматическое создание базы, открытие страниц и запуск тестов. Эта последовательность выполняется на чистой машине GitHub Actions при каждой отправке изменений.

Найденные в ходе ручных проверок ошибки и их исправления приведены в таблице 7.

Таблица 7 – Найденные ошибки и исправления

№	Ошибка	Причина	Исправление	Повторная проверка
1	Запрос к API с неверной датой («вчера») или пустым телом возвращал английский ответ «One or more validation errors occurred»	использовался стандартный ответ ASP.NET Core на ошибки привязки данных	задан собственный ответ { error } на русском (ValidationMessages. ApiResponse)	HTTP 400: {"error":"Неверный формат данных в запросе, проверьте поля: date. Даты передаются в виде \"ГГГГ-ММ-ДД\""}
2	При подмене значения в форме выводилось английское сообщение «The value 'abc' is not valid…» вместе с русским	стандартные сообщения привязки данных ASP.NET Core	заданы русские сообщения (ValidationMessages. Configure); при ошибке привязки сервис не вызывается	Значение «abc» не подходит для поля «Оборудование»

Оставшиеся ограничения: нет входа в систему и ролей (не предусмотрено заданием, модулем пользуется один ответственный); защита от одновременной выдачи действует в пределах одного запущенного экземпляра приложения; стили Bootstrap подключаются из сети.

Выводы по тестированию. Все 69 автоматических проверок пройдены, все ручные проверки дали ожидаемый результат. Главное правило и граничные случаи – повторная выдача, выдача в день возврата, возврат раньше выдачи, возврат в состоянии «Требует ремонта» – подтверждены тестами. Найденные ошибки исправлены, проверки повторены.

Выводы по разделу 3

Реализованы все обязательные функции темы: справочники оборудования и сотрудников, выдача и возврат с датой операции и состоянием оборудования, изменение состояния и списание, список оборудования у сотрудников и история операций с фильтрами. Предметная логика сосредоточена в сервисах и проверена 69 автоматическими тестами; интерфейс, API и сохранение данных

после перезапуска проверены в браузере. Модуль разворачивается одной командой, база создаётся автоматически.

ЗАКЛЮЧЕНИЕ

В курсовой работе разработан модуль учёта выдачи оборудования сотрудникам. Цель работы достигнута, что подтверждается решением каждой задачи.

Предметная область проанализирована в разделе 1: выявлены проблемы ручного учёта – повторная выдача, потеря сведений о состоянии, невозвращённое оборудование уволенных сотрудников; рассмотрены аналоги; сформулированы семь функциональных требований и восемь ограничений; выбраны средства разработки – C#, .NET 10, ASP.NET Core MVC, EF Core с SQLite, xUnit и Git.

Проектирование выполнено в разделе 2: модель данных из трёх сущностей; определение держателя оборудования по истории операций без отдельного поля в базе; условия выдачи и проверки даты; диаграмма состояний единицы оборудования; трёхслойная архитектура и алгоритм выдачи с защитой от одновременных операций.

Реализация описана в подразделах 3.1–3.5: все функции задания работают, недопустимые операции отклоняются с понятными сообщениями, неизвестная запись даёт страницу «Запись не найдена», данные сохраняются между запусками.

Тестирование выполнено в подразделе 3.6: 69 автоматических проверок и ручные проверки в браузере пройдены, две найденные ошибки исправлены. Подготовлены инструкция запуска, инструкция пользователя и материалы к защите.

Перспективы развития: вход в систему и роли (ответственный, руководитель), печать акта приёма-передачи, напоминания о долгом нахождении оборудования у сотрудника, учёт ремонтов с исполнителем и стоимостью, выгрузка отчётов в Excel.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. О бухгалтерском учете : Федеральный закон № 402-ФЗ : [принят Государственной думой 22 ноября 2011 года : одобрен Советом Федерации 29 ноября 2011 года]. – Текст : электронный // КонсультантПлюс. – URL: https://www.consultant.ru/document/cons_doc_LAW_122855/ (дата обращения: 12.11.2026).

2. Об утверждении Федерального стандарта бухгалтерского учета ФСБУ 6/2020 «Основные средства» : Приказ Министерства финансов Российской Федерации от 17 сентября 2020 г. № 204н. – Текст : электронный // Гарант.ру. – URL: https://www.garant.ru/products/ipo/prime/doc/74665146/ (дата обращения: 12.11.2026).

3. Российская Федерация. Законы. Трудовой кодекс Российской Федерации : Федеральный закон № 197-ФЗ : [принят Государственной думой 21 декабря 2001 года : одобрен Советом Федерации 26 декабря 2001 года]. – Текст : электронный // КонсультантПлюс. – URL: https://www.consultant.ru/document/cons_doc_LAW_34683/ (дата обращения: 12.11.2026).

4. Lock, A. ASP.NET Core in Action / A. Lock. – 2nd ed. – Shelter Island : Manning, 2021. – ISBN 978-1-61729-830-1.

5. Smith, J. P. Entity Framework Core in Action / J. P. Smith. – 2nd ed. – Shelter Island : Manning, 2021. – ISBN 978-1-61729-836-3.

6. Прайс, М. Дж. C# 10 и .NET 6. Современная кросс-платформенная разработка / М. Дж. Прайс ; перевод с английского С. Черникова. – 6-е изд. – Санкт-Петербург : Питер, 2023. – 848 с. – (Для профессионалов). – ISBN 978-5-4461-2249-3.

7. Khorikov, V. Unit Testing: Principles, Practices, and Patterns / V. Khorikov. – Shelter Island : Manning, 2020. – ISBN 978-1-61729-627-7.

8. Chacon, S. Pro Git / S. Chacon, B. Straub. – 2nd ed. – Текст : электронный // Git. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 12.11.2026).

9. Документация по .NET. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/ru-ru/dotnet/ (дата обращения: 12.11.2026).

10. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : издание официальное : утвержден и введен в действие Постановлением Государственного комитета СССР по управлению качеством продукции и стандартам от 26.12.90 № 3294 : дата введения 1992-01-01. – Москва : Стандартинформ, 2010. – 22 с.

11. Overview of ASP.NET Core MVC. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/aspnet/core/mvc/overview (дата обращения: 12.11.2026).

12. Entity Framework Core. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/ef/core/ (дата обращения: 12.11.2026).

Приложение А

Листинг тестов определения держателя оборудования

Файл EquipmentIssue.Tests/OnHandCalculatorTests.cs

```
using EquipmentIssue.Models;
using EquipmentIssue.Services;

namespace EquipmentIssue.Tests;

public class OnHandCalculatorTests
{
    private static readonly DateTime D = TestEnv.Today;

    private static EquipmentOperation Op(int id, int equipmentId, OperationType
type, int daysAgo, int? employeeId = null) =>
        new() { Id = id, EquipmentId = equipmentId, Type = type, Date =
D.AddDays(-daysAgo), EmployeeId = employeeId };

    [Fact]
    public void Держатель_НетОпераций_НаСкладе()
    {
        Assert.Null(OnHandCalculator.CurrentHolderId(new
List<EquipmentOperation>()));
    }

    [Fact]
    public void Держатель_ПослеВыдачи_Сотрудник()
    {
        var ops = new[] { Op(1, 1, OperationType.Issue, 5, employeeId: 7) };
        Assert.Equal(7, OnHandCalculator.CurrentHolderId(ops));
    }

    [Fact]
    public void Держатель_ВыдачаИВозврат_НаСкладе()
    {
        var ops = new[] { Op(1, 1, OperationType.Issue, 5, 7), Op(2, 1,
OperationType.Return, 2, 7) };
        Assert.Null(OnHandCalculator.CurrentHolderId(ops));
    }

    [Fact]
    public void Держатель_ПовторнаяВыдача_ВтораяСотрудник()
    {
        var ops = new[] { Op(1, 1, OperationType.Issue, 9, 7), Op(2, 1,
OperationType.Return, 5, 7), Op(3, 1, OperationType.Issue, 2, 8) };
        Assert.Equal(8, OnHandCalculator.CurrentHolderId(ops));
    }

    [Fact]
    public void Держатель_ПорядокПоДатеАНеПоСписку()
    {
        // Операции переданы не по порядку: возврат (позже) стоит раньше выдачи
        var ops = new[] { Op(2, 1, OperationType.Return, 1, 7), Op(1, 1,
OperationType.Issue, 4, 7) };
        Assert.Null(OnHandCalculator.CurrentHolderId(ops));
    }

    [Fact]
    public void Держатель_ВозвратИВыдачаВОдинДень_ПоНомеруОперации()
    {
```

```
        var ops = new[] { Op(1, 1, OperationType.Issue, 3, 7), Op(2, 1,
OperationType.Return, 0, 7), Op(3, 1, OperationType.Issue, 0, 8) };
        Assert.Equal(8, OnHandCalculator.CurrentHolderId(ops));
    }

    [Fact]
    public void Держатель_СменаСостоянияНеВлияет()
    {
        var ops = new[] { Op(1, 1, OperationType.ConditionChange, 3), Op(2, 1,
OperationType.Issue, 2, 7) };
        Assert.Equal(7, OnHandCalculator.CurrentHolderId(ops));
    }

    [Fact]
    public void ТекущиеВыдачи_НесколькоЕдиниц_ТолькоВыданные()
    {
        var ops = new[]
        {
            Op(1, 1, OperationType.Issue, 10, 7),
            Op(2, 2, OperationType.Issue, 9, 8),
            Op(3, 1, OperationType.Return, 5, 7),
            Op(4, 3, OperationType.Issue, 1, 7)
        };
        var current = OnHandCalculator.CurrentIssues(ops);
        Assert.Equal(new[] { 2, 3 }, current.Keys.OrderBy(k => k));
        Assert.Equal(8, current[2].EmployeeId);
    }

    [Theory]
    [InlineData(0, 0)]
    [InlineData(1, 1)]
    [InlineData(30, 30)]
    [InlineData(-2, 0)]   // дата в будущем (ошибка данных) не даёт
отрицательного числа
    public void ДнейНаРуках(int daysAgo, int expected)
    {
        Assert.Equal(expected, OnHandCalculator.DaysOnHand(D.AddDays(-daysAgo),
D));
    }
}
```

Приложение Б

Листинг правил предметной области

Файл Services/OperationRules.cs

```
using EquipmentIssue.Models;

namespace EquipmentIssue.Services;

public static class OperationRules
{
    public const int MaxCommentLength = 200;
    public const int MinWriteOffReasonLength = 5;

    public static string? ValidateDate(DateTime? date, DateTime today, DateTime?
lastOperationDate)
    {
        if (date == null) return "Укажите дату операции";
        if (date.Value.Date > today.Date) return "Дата операции не может быть в
будущем";
        if (lastOperationDate != null && date.Value.Date <
lastOperationDate.Value.Date)
            return $"Дата операции не может быть раньше последней операции по
этому оборудованию ({lastOperationDate:dd.MM.yyyy})";
        return null;
    }

    public static string? CheckIssue(Equipment equipment, Employee employee,
EquipmentOperation? currentIssue)
    {
        if (equipment.IsWrittenOff)
            return $"Оборудование «{equipment.Name}» списано — выдача
невозможна";
        if (currentIssue != null)
        {
            var holder = currentIssue.Employee != null ?
NameFormat.Short(currentIssue.Employee.FullName) : "другого сотрудника";
            return $"Оборудование «{equipment.Name}» уже выдано: находится у
{holder} с {currentIssue.Date:dd.MM.yyyy}. Сначала оформите возврат";
        }
        if (equipment.Condition == EquipmentCondition.NeedsRepair)
            return $"Оборудование «{equipment.Name}» требует ремонта — выдача
запрещена до ремонта";
        if (!employee.IsActive)
            return $"Сотрудник {NameFormat.Short(employee.FullName)} уволен —
выдача невозможна";
        return null;
    }

    public static string? CheckReturn(Equipment equipment, EquipmentOperation?
currentIssue, EquipmentCondition? condition, string? comment)
    {
        if (currentIssue == null)
            return $"Оборудование «{equipment.Name}» не числится выданным —
возвращать нечего";
        if (condition == null || !Enum.IsDefined(condition.Value))
            return "Укажите состояние оборудования при возврате";
        if (condition == EquipmentCondition.NeedsRepair &&
string.IsNullOrWhiteSpace(comment))
            return "Опишите неисправность в комментарии: оборудование
возвращается в состоянии «Требует ремонта»";
```

```
        return null;
    }

    public static string? CheckComment(string? comment) =>
        comment != null && comment.Trim().Length > MaxCommentLength
            ? $"Комментарий — не длиннее {MaxCommentLength} символов"
            : null;
}
```